Component A: Staff Interview

Interviewee: Dorothy, Program Coordinator — Student Purchasing

Direct Quotes

- "Too much communication, need feedback" — Dorothy expressed that the current process generates excessive back-and-forth, and she needs a system that can automatically update students on order status.
- "Easier for students and her" — Her ideal solution benefits both sides, not just the coordinator.

Surprises

- Dorothy only processes purchase requests once a week, but order status changes (back-orders, delays, returns) happen continuously throughout the week — meaning students have no real-time visibility into what is happening with their order between submissions.
- The volume is significant: 20–100 orders per month, yet the entire tracking system relies on a manually maintained Excel spreadsheet.

Current Workarounds

- Students submit requests via a Google Form
- Dorothy manually transfers entries into a shared Excel spreadsheet, separated into Amazon and non-Amazon tabs
- A summary tab aggregates all previous tabs and is continuously updated
- For group projects: each team appoints a CFO who can place orders directly using the school's Amazon account
- For individual projects: all requests must go through Dorothy via the form
- For non-Amazon suppliers: Dorothy must manually create vendor accounts or handle additional details, and physically check packages and emails to confirm receipt
- Receipts are collected in both physical and digital formats
- Instructor approval is required before any order is placed

User Journey Map

Step	Action	Time	Where Info Lives	What Could Go Wrong
1	Student fills out Excel with item details	~5 min	Excel spreadsheet	Incomplete fields, broken product links
2	Dorothy sorts requests: Amazon vs. non-Amazon	~mins	Excel + email	Non-Amazon requires extra account setup
3	Amazon orders placed via school account; non-Amazon handled via email/physical check	Hours	Amazon account / inbox	Account access issues, lost packages

4	Order status changes (back-order, delay, return) , Dorothy must notify each student manually	Ongoing throughout week	Dorothy's memory + Teams + Excel	Students not informed, repeated inquiries
5	Items arrive; Dorothy reconciles receipts and updates "Actual Price" column in Excel	Weekly	Physical receipts + Excel	Lost receipts, price discrepancies
6	Students follow up asking about order status	Anytime	Teams / in-person	Repetitive communication, coordinator overload

What Success Looks Like (in Dorothy's words)
Students can see their order status on their own, without needing to ask her
She can instantly see who submitted what, what they want, and which class it belongs to
The system reduces back-and-forth communication around delays, back-orders, and returns

Problem Statement
"When GIX students need to submit and track purchase requests, they currently fill out a Google Form and rely on Dorothy to manually update a shared Excel spreadsheet, which causes students to have no visibility into their order status and forces Dorothy to handle constant back-and-forth communication about delays, back-orders, and returns."

Component B: Lab

AI Usage Log

Interaction 1

- **Prompt given:** "Build a Streamlit app called 'GIX Purchase Request Tracker' that helps GIX students submit and track purchase requests, and helps the coordinator Dorothy manage them easily. The app should have two main sections selectable from a sidebar: 1. Submit a Request — a form where students can fill in their name, class name, team number, supplier, item name, quantity, unit price, product link, notes, and instructor approval status. When submitted, save the entry to a local CSV file called requests.csv. 2. View All Requests — a table that reads from requests.csv and displays all submitted requests with filters by supplier and instructor approval status, and a summary count at the top."
- **What Cursor produced:** A complete 261-line app.py with two-page navigation, a submission form saving to CSV, and a coordinator view with summary metrics, multi-select filters, and an editable status update section.
- **First try?** No. The app crashed with AttributeError: module 'streamlit.column_config' has no attribute 'SelectColumn'. I pasted the error back into Cursor and it fixed the status update section by replacing SelectColumn with a standard selectbox approach.

Interaction 2

- **Prompt given:** "Add a text search bar at the top of the 'View All Requests' section, above the filters. It should search across student_name, item_name, and class_name columns in real time as the user types."
- **What Cursor produced:** Added a "Search requests" text input above the supplier/approval filters. Typing filters the displayed table in real time across student name, item name, and class name.
- **First try?** Yes. The search bar appeared immediately after accepting the changes.

Interaction 3

- **Prompt given:** "Add an 'About' option to the existing sidebar radio navigation. The About page should show: a title 'About This App', a description of the project: this app was built for TECHIN 510 to help GIX students submit purchase requests and help coordinator Dorothy track and manage orders, replacing the current Google Form and Excel workflow. A link to the GIX website: https://www.gix.uw.edu. The developer's name and course info."
- **What Cursor produced:** Added a third navigation option "About" to the sidebar radio buttons. The About page displays a project description, GIX website link, developer name, and course information pulled from constants defined at the top of the file (DEVELOPER_NAME, DEVELOPER_COURSE).
- **First try?** Yes. The About page appeared immediately after accepting the changes. I then manually updated DEVELOPER_NAME from "Your Name" to my actual name.

Accessibility Baseline Check Results

Check 1 — Color Contrast:
Text color: #FAFAFA (rgb 250,250,250) on background #0E1117 (rgb 14,17,23). Contrast ratio: ~19.7:1. Result: Pass (required minimum: 4.5:1).
Check 2 — Semantic Headings:
App uses exactly one st.title() for the main title. Section headings use st.header() and st.subheader(). Found one instance of st.markdown("### Navigation") used as a heading — manually replaced with st.subheader("Navigation"). Result: Pass

Reflection

1. What surprised you about AI-assisted coding?

Honestly I didn't expect it to work this fast. I just described what I wanted in plain English and Cursor built the whole app — form, table, filters, everything. That felt kind of unreal. The "wow" moment was when I ran it for the first time and it actually opened in the browser and worked. The "wait that's not right" moment came right after when the coordinator page crashed with an error I had never seen before. So yeah, impressive but you still have to pay attention.

2. What did the AI get wrong?

Cursor used a Streamlit feature called SelectColumn that doesn't exist in my installed version, so the whole View All Requests page crashed immediately. I copied the error message and pasted it back into Cursor and it fixed it on the first try. I also caught during the accessibility check that it used st.markdown("### Navigation") as a heading instead of st.subheader() — that one I fixed myself without asking the AI, which made me feel like I actually understood something.

3. Could you explain your code?

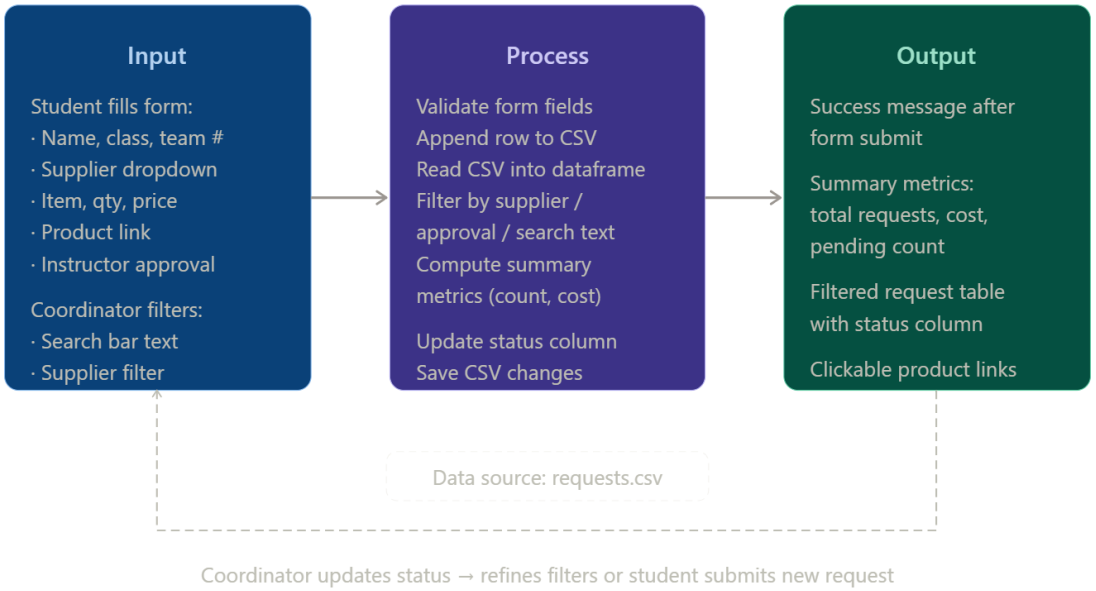
I think I could explain the search bar part. When you type in the search box, the code checks if that text appears anywhere in the student name, item name, or class name columns. It uses .str.contains() with case=False so it doesn't matter if you type uppercase or lowercase. Then it just shows the filtered rows. I know this because I asked Cursor to explain it line by line and then tested it myself by typing different things to see what happened.

4. What did you learn from the interview?

Before talking to Dorothy I would have probably just built a basic form where students submit requests and that's it. But she kept saying the hard part isn't receiving the requests — it's everything after. Tracking status, notifying students about delays, dealing with back-orders. That completely changed what I built. The status update section and the summary metrics at the top came directly from that conversation. Without the interview I would have solved the wrong problem.

Component C: System Architecture & Design

Diagramming Exercise: Draw I/P/O for Your Streamlit App



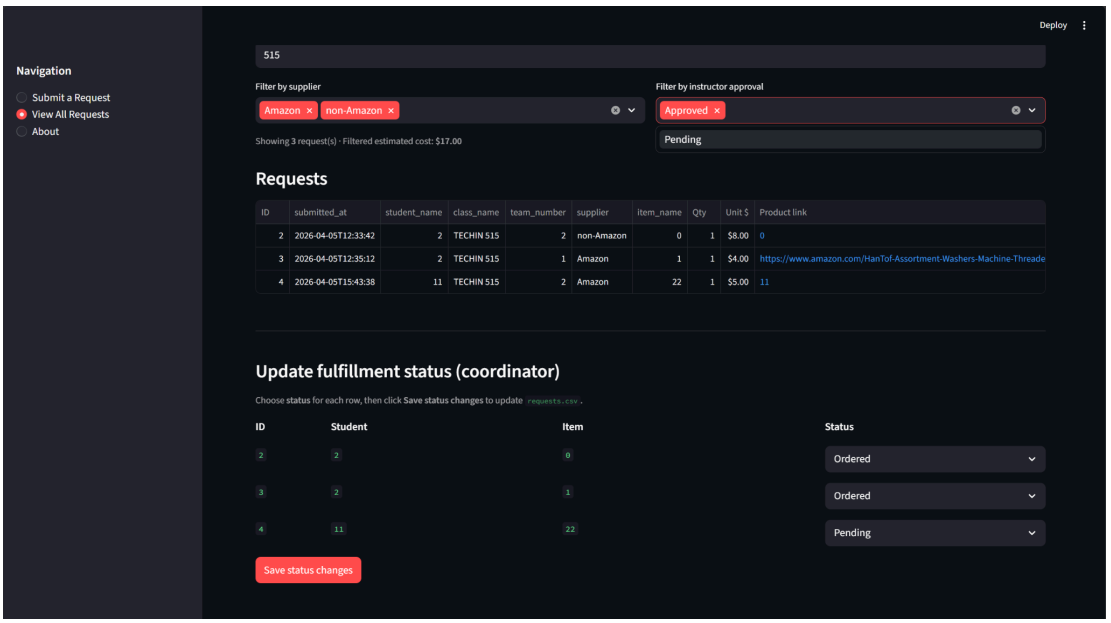
Design Decision Log

Field	Your Entry
Decision	Put all code in a single file: app.py
Alternatives considered	Split into multiple files: e.g. pages/submit.py, pages/view.py, utils/data.py for data handling

Why I chose this	Cursor generated a single file by default. For a 261-line app, one file is still easy to navigate. The assignment scope is small enough that splitting would add complexity without benefit
Trade-off	If I add 5+ more features, a single file becomes harder to read. Functions would get harder to find and the file would scroll endlessly
When would I choose differently?	If the app grew beyond ~500 lines, or if multiple developers were working on it simultaneously, splitting into modules would make collaboration and maintenance much easier

Component D: Testing & Validation

#	Feature Tested	Action Taken	Expected Result	Actual Result	Pass/Fail
1	Submit a Request form	Filled in all required fields (name, class, supplier, item, quantity, price) and clicked "Submit request"	Success message appears and new entry is saved	Green success message displayed: "Request submitted. Thank you — Dorothy can review it under View All Requests." New entry (ID 4) appeared in the table immediately	Pass
2	Real-time search bar	Typed "515" in the Search requests box on View All Requests page	Table filters to show only rows where class name contains "515"	Table updated instantly showing 3 matching rows (IDs 2, 3, 4), all with class_name "TECHIN 515". Row count updated to "Showing 3 request(s)"	Pass
3	App startup and summary metrics	Launched app with streamlit run app.py and navigated to View All Requests	App loads without errors and displays correct summary metrics	App opened successfully. Summary showed: Total requests = 4, Total estimated cost = $22.00, Fulfillment still pending = 1. No errors in terminal or browser	Pass



Quality Gate Checklist

☑ Smoke test table completed with 3 tested features

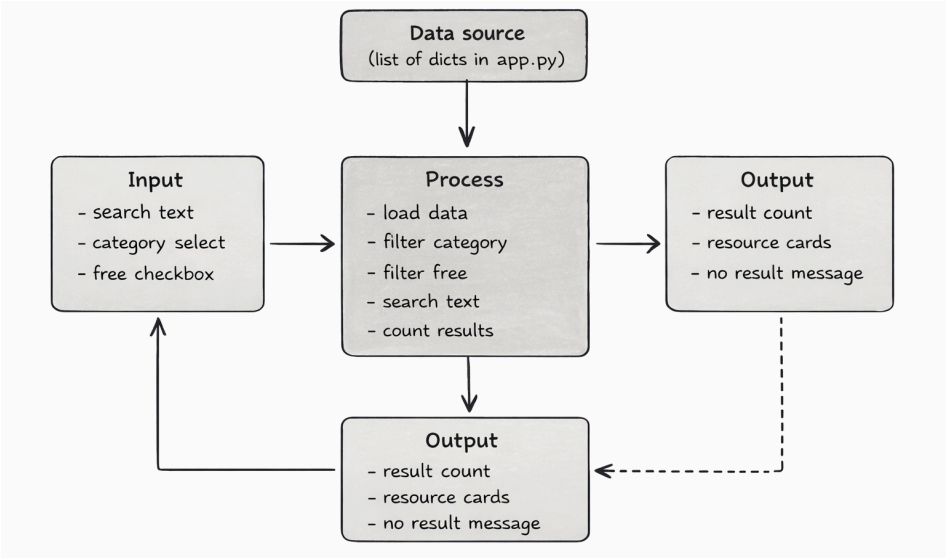
☑ All tests passed, no failures to document

☑ Screenshot of running app included (View All Requests page with search active)

☑ Accessibility baseline results recorded (color contrast pass 19.7:1, semantic headings pass)

Component E: Applied Challenge — The GIX Wayfinder

I/P/O diagram



Edge Case 1

When a user types a search term that matches no resource names or descriptions, the app must display a helpful message instead of showing an empty table. This matters because a blank screen gives no feedback — the user doesn't know if the app broke or if the resource genuinely doesn't exist. Tested by typing "xyz123" in the search bar. Result: friendly "No resources found" message appeared correctly.

Edge Case 2

When a user removes all categories from the multiselect filter, no resources match any category. The app must handle this gracefully rather than crashing. This matters because an empty filter state is easy to accidentally trigger and should still show a clear empty state message. Tested by clicking X on every category tag. Result: result count dropped to 0 and empty state message displayed correctly.

Assert Statement

The assert statement is located in wayfinder/app.py, immediately after the resources list is defined. It verifies that every dictionary in the list contains the three required fields: name, category, and location.

Prompt Log

Initial prompt: "Build a Streamlit app called GIX Campus Wayfinder... Store resources as a list of Python dictionaries... Include at least 12 real-sounding GIX campus resources..."
Refinement: Added the requirement to handle the empty state explicitly: "if no results match, show a friendly message like 'No resources found. Try a different search term or category.'"
What changed and why: The first prompt focused on the happy path — showing results. The refinement added the empty state because without it, the app would show nothing when filters were too restrictive, leaving users confused about whether the app was working correctly.